

dwmwhat

structured compile-time string literals in C++ for package identification

Chapter 1

Introduction

dwmwhat(1) is a command line utility that is very similar to *what(1)* from SCCS. It searches one or more files for strings that start with "*@(#)*" and displays them on standard output. Such strings can be embedded in binaries to represent version information.

The *dwmwhat* package includes a C++ class template `Dwm::What::Info` that can be used to create structured versions of these strings from string literals. This class template is intended to be instantiated to identify the package name, version and other information inside a string that will be visible in compiled code. It is constructed at compile time, and because it is structured, *dwmwhat(1)* can parse it and hence present it decomposed into its components (as it does when the `-j` command line option is used).

For example, let's say you create a trivial program in `myprog.cc` as seen in [Listing 1.1](#):

```
1 #include <iostream>
2 #include "DwmWhatInfo.hh"
3
4 //-----
5 Dwm::What::Info myproginfo(DWM_WHAT_TYPE_EXE, DWM_WHAT_STATUS_REL, "myprog",
6                             "1.0.0", DWM_WHAT_SYM_COPYRIGHT " Jane Doe 2025",
7                             "hello!");
8
9 //-----
10 int main(int argc, char *argv[])
11 {
12     std::cout << myproginfo.data_view() << '\n';
13     std::cout << myproginfo.as_json() << '\n';
14     return 0;
15 }
```

Listing 1.1: `myprog.cc`

If you compile and link this program, *dwmwhat(1)* will report this:

```
% dwmwhat myprog
🤖 ✅ myprog 1.0.0 © Jane Doe 2025 hello!
🤖 # ✅ dwmwhat 1.0.0 © Daniel McRobb 2025 mcplex.net
```

And can also report in JSON form:

```
% dwmwhat -j myprog
[{"file":"myprog","infos":[{"copyright":"© Jane Doe 2025","name":"myprog",
"other":"hello!","status":"✅","type":"🤖","version":"1.0.0", "view":"@(#)
🤖 ✅ myprog 1.0.0 © Jane Doe 2025 hello!"}, {"copyright": "© Daniel
McRobb 2025","name":"dwmwhat","other":"mcplex.net","status":"✅",
"type":"🤖#","version":"1.0.0","view":"@(#) 🤖# ✅ dwmwhat 1.0.0 © Daniel
McRobb 2025 mcplex.net"}]}
```

1.1 Under the hood

Let's look at what's going on under the hood.

We're building a null-terminated string literal at compile time (the `Dwm::What::Info` constructor is `constexpr`). Note that the only reason we need the null termination is to allow `dwmwhat(1)` and similar tools to find the end of the string in a binary.

The constructor call:

```
Dwm::What::Info(DWM_WHAT_TYPE_EXE, DWM_WHAT_STATUS_REL, "myprog", "1.0.0",
                DWM_WHAT_SYM_COPYRIGHT " Jane Doe 2025", "hello!");
```

Produces a string as shown in [Figure 1.1](#).

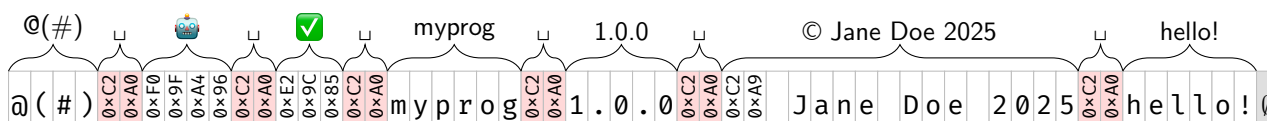


Figure 1.1: `Dwm::What::Info` layout

All we've done here is concatenate *segments* of string literals, separated by a fixed *delimiter*. Some of our string literals are Unicode code points in UTF-8, such as 🚗 ("`\xF0\x9F\xA4\x96`") and ✅ ("`\xE2\x9C\x85`"), for which I created macros in `DwmWhatMacros.hh`. And for the copyright segment, I concatenate `DWM_WHAT_SYM_COPYRIGHT` ("`\xC2\xA9`") with " Jane Doe 2025".

1.1.1 Choice of delimiter

The delimiter is key to being able to decompose the string in tools like `dwmwhat(1)`. It needs to be distinct, and never appear inside a *segment*.

It turns out that this was probably the most time-consuming part of doing this work. Let's start with what I wanted in a delimiter.

Desirable delimiter features

1. Painless to disallow in the user-specified content (*segments*). We want something that the user will never need.
2. Does not clutter the display of the entire literal (including the delimiters).
3. Human readability of strings (*segments* concatenated with *delimiters*, together) in binary files.
4. Machine readability of strings in binary files (the ability to decompose into segments), without compromising the human readability.

1.2 Delimiter is Unicode *No-Break Space (NBSP)* (U+00A0)

I've chosen Unicode *No-Break Space (NBSP)* as the delimiter, since it serves the needs well. For a human it just looks like a single space, while it can be distinguished from an ASCII space (`0x20`) when parsing. The UTF-8 encoding for *NBSP* is "`\xC2\xA0`", i.e. `{0xC2, 0xA0}`.

When you want single spaces in any of your segments, just use ASCII spaces (`0x20`) and not *NBSP*.

Chapter 2

Typical Use

2.1 Executable

In an executable, just declare a static constexpr `Dwm::What::Info` in the same translation unit as `main()`, preferably in a unique namespace. You may want to mark your instance with `__attribute__((used))` to prevent the toolchain from optimizing it out of existence due to lack of use. For example:

```
namespace MyNamespace {
    namespace myproginfo {
        static constexpr __attribute__((used))
            info(DWM_WHAT_TYPE_EXE, DWM_WHAT_STATUS_REL, "myprog", "1.0.0",
                DWM_WHAT_SYM_COPYRIGHT " My Name 2025", "my program");
    }
}
```

2.2 Compiled library

In one of your source files, just declare a static constexpr `Dwm::What::Info`, preferably in a unique namespace. You may want to mark your instance with `__attribute__((used))` to prevent the toolchain from optimizing it out of existence due to lack of use. For example:

```
namespace MyNamespace {
    namespace mylibinfo {
        static constexpr __attribute__((used))
            info(DWM_WHAT_TYPE_LIB, DWM_WHAT_STATUS_REL, "mylib", "1.0.0",
                DWM_WHAT_SYM_COPYRIGHT " My Name 2025", "my library");
    }
}
```

2.3 Header-only library

In one of your header files, declare an inline constexpr `Dwm::What::Info`, preferably in a unique namespace. You may want to mark your instance with `__attribute__((used))` to prevent the toolchain from optimizing it out of existence due to lack of use. For example:

```
namespace MyNamespace {
    namespace myhdrinfo {
        inline constexpr __attribute__((used))
            info(DWM_WHAT_TYPE_HDR, DWM_WHAT_STATUS_REL, "myheaderlib", "1.0.0",
                DWM_WHAT_SYM_COPYRIGHT " My Name 2025", "my header library");
    }
}
```

2.3.1 Caveats

It's worth noting that the usual caveats apply here. Despite the fact that `inline constexpr` is the recommended means of producing a single instance of such a constant in a linked program, ODR (One Definition Rule) violations can rear their head in difficult-to-diagnose ways. Just be aware: today's linkers aren't good at identifying ODR violations. If two object files are compiled with different versions of your header and then linked, the linker is going to pick one of your `inline constexpr` variables, despite the fact that they might not match in content or even type. It's also NDR (No diagnostic Required).

This is no different than other things that can go wrong with ODR violations when two object files are compiled different versions of a header file. Just be aware. My recommendation is to name your instance of `Dwm::What::Info` differently for each release, and provide an accessor that returns a reference to that instance. Then the linker will see two identifiers instead of one, and will keep both. Which will at least allow you to see that you have more than one version linked into a binary.

I mention this because in the case of `Dwm::What::SegmentedLiteral` and `Dwm::What::Info`, a declaration is typically stamping out an instance of a new type. The template is parameterized by the number and size of the segments, so any change to either of these will yield a different type, not just different content. The linkers of today don't seem to use this information to flag ODR violations.

Chapter 3

Class templates in `Dwm::What` namespace

There are two class templates of interest in the *dwmwhat* package: `Dwm::What::SegmentedLiteral` and `Dwm::What::Info`. We saw simple use of the `Dwm::What::Info` class template in the introduction, and it is likely the only class template most users will use.

However, `Dwm::What::Info` inherits from `Dwm::What::SegmentedLiteral`, and I'll introduce them in base to derived order. `Dwm::What::SegmentedLiteral` has no constraints on number of segments or choice of delimiter (an empty delimiter is permissible), and all of the segments are user defined.

3.1 `Dwm::What::SegmentedLiteral`

`Dwm::What::SegmentedLiteral` builds a string literal from a given delimiter (placed between segments) and one or more segments (given as string literals). All arguments to the constructor are string literals, and used to deduce the template parameters.

An empty delimiter is allowed, as are empty segments.

3.1.1 Synopsis

```
template <std::size_t DelimLen, std::size_t FirstLen, std::size_t ... SegLen>
class SegmentedLiteral
{
public:
    constexpr SegmentedLiteral(const char (&delim)[DelimLen],
                               const char (&firstSeg)[FirstLen],
                               const char (&... moreSegs)[SegLen]);
    constexpr std::string_view view() const noexcept;
    constexpr std::size_t num_segments() const noexcept;
    constexpr std::string_view nth(std::size_t n) const noexcept;
};
```

3.1.2 Member functions

Constructor

The `Dwm::What::SegmentedLiteral` constructor requires a delimiter (a string literal) and at least one segment. All segments are given as string literals.

delim

The delimiter that will be placed between segments, which may be empty.

firstSeg

The first segment.

moreSegs

The remaining segments.

Accessors

view()

returns a `std::string_view` of the entire constructed string literal, sans the terminating null.

num_segments()

returns the number of segments.

nth(std::size_t n)

returns the *n*th segment (0 indexed).

3.1.3 Examples

```
Dwm::What::SegmentedLiteral(" ", "@(#)", "DwmWhat", "1.0.0",
                             "Copyright Daniel McRobb 2025", "mcplex.net");
```

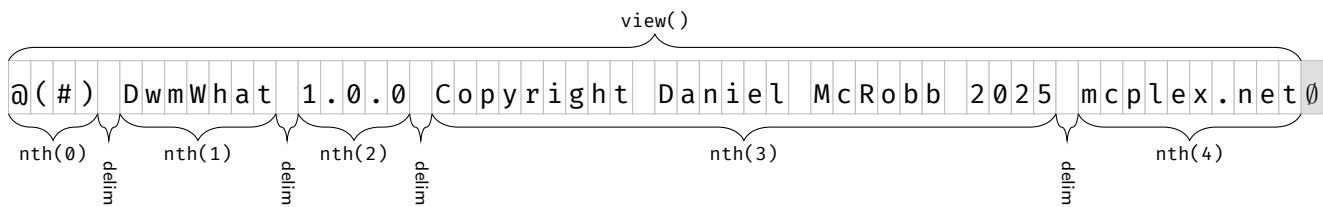


Figure 3.1: `Dwm::What::SegmentedLiteral` layout

```
Dwm::What::SegmentedLiteral("|", "@(#)", "DwmWhat", "1.0.0",
                             "Copyright Daniel McRobb 2025", "mcplex.net");
```

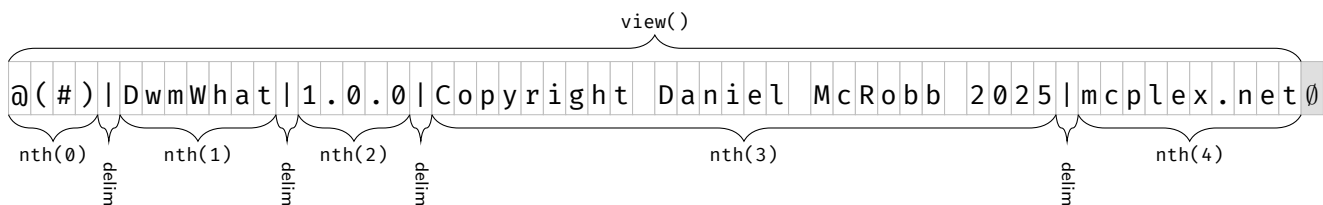


Figure 3.2: `Dwm::What::SegmentedLiteral` layout

3.2 Dwm::What::Info

Dwm::What::Info is just a Dwm::What::SegmentedLiteral with seven segments and a specific delimiter (not user-specified). The first segment ("@(#)") is automatically added, and named accessors are provided for the remaining six segments.

3.2.1 Synopsis

```
template <std::size_t TypeLen, std::size_t StatusLen, std::size_t NameLen,
          std::size_t VersionLen, std::size_t CopyrightLen, std::size_t OtherLen>
class Info
    : public SegmentedLiteral<sizeof(DWM_WHAT_DELIM), sizeof("@(#)"), TypeLen,
                             StatusLen, NameLen, VersionLen, CopyrightLen, OtherLen>
{
public:
    constexpr Info(const char (&type)[TypeLen], const char (&status)[StatusLen],
                   const char (&name)[NameLen], const char (&version)[VersionLen],
                   const char (&copyright)[CopyrightLen], const char (&other)[OtherLen]);

    constexpr std::string_view type() const noexcept;
    constexpr std::string_view status() const noexcept;
    constexpr std::string_view name() const noexcept;
    constexpr std::string_view version() const noexcept;
    constexpr std::string_view copyright() const noexcept;
    constexpr std::string_view other() const noexcept;
    constexpr std::string_view data_view() const noexcept;
    constexpr std::string_view view() const noexcept;
};
```

3.2.2 Constructor

constructor arguments

type

The type of the package. By convention, one or more of DWM_WHAT_TYPE_EXE, DWM_WHAT_TYPE_LIB, DWM_WHAT_TYPE_HDR and DWM_WHAT_TYPE_DOC. These are just macros that expand to string literals. See [Table 6.1](#).

status

The status of the package. By convention, one of DWM_WHAT_STATUS_REL, DWM_WHAT_STATUS_RC or DWM_WHAT_STATUS_DEV. These are just macros that expand to string literals. See [Table 6.1](#).

name

The name of the package.

version

The version of the package.

copyright

The copyright.

other

Any other additional information.

3.2.3 Accessors

type()

Returns the type of the package. This is the same that was provided as the first argument to the constructor.

status()

Returns the status of the package. This is the same that was provided as the second argument to the constructor.

name()

Returns the name of the package. This is the same that was provided as the third argument to the constructor.

version()

Returns the version of the package. This is the same that was provided as the fourth argument to the constructor.

copyright()

Returns the copyright. This is the same that was provided as the fifth argument to the constructor.

other()

Returns any additional information that was provided. This is the same as the sixth (final) argument to the constructor.

3.2.4 Equivalence to `Dwm :: What :: SegmentedLiteral`

The following will produce the equivalent string literal. This makes it obvious that `Dwm :: What :: Info` is just a `Dwm :: What :: SegmentedLiteral` with a specific delimiter ("`\xC2\xA0`") and an automatically added first segment ("`@(#)`").

```
SegmentedLiteral("\xC2\xA0", "@(#)",
    DWM_WHAT_TYPE_LIB, DWM_WHAT_STATUS_REL, "DwmWhat",
    "1.0.0", "Daniel McRobb 2025", "mcplex.net");

Info(DWM_WHAT_TYPE_LIB, DWM_WHAT_STATUS_REL, "DwmWhat",
    "1.0.0", "Daniel McRobb 2025", "mcplex.net");
```

3.2.5 Examples

This call to the `Dwm::What::Info` constructor:

```
Dwm::What::Info(DWM_WHAT_TYPE_EXE DWM_WHAT_TYPE_HDR, DWM_WHAT_STATUS_REL,
               "DwmWhat", "1.0.0", "Daniel McRobb 2025", "mcplex.net");
```

Would produce a 67 byte string literal at compile time with the layout shown in [Figure 3.3](#). Note this figure shows what would be returned by accessors inherited from the `Dwm::What::SegmentedLiteral` base class (in black) and the accessors from the `Dwm::What::Info` class (in blue).

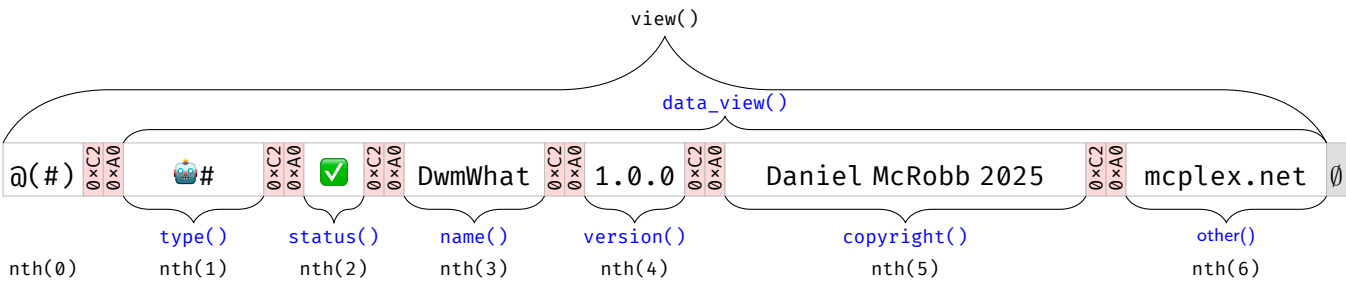


Figure 3.3: `Dwm::What::Info` layout

The 67 bytes:

@(#)	4 bytes
delimiter	2 bytes
type	7 bytes
delimiter	2 bytes
status	3 bytes
delimiter	2 bytes
name	7 bytes
delimiter	2 bytes
version	5 bytes
delimiter	2 bytes
copyright	18 bytes
delimiter	2 bytes
other	10 bytes
null	1 byte
Total	67 bytes

Note that we also store 7 additional bytes to hold the segment lengths (one byte per segment). If the total length of your string exceeds 255 bytes, it will be 14 bytes (two bytes per segment).

`dwmwhat` would print this when scanning a compiled program containing such an instance of `Dwm::What::Info`:

```
🤖# ✅ DwmWhat 1.0.0 Daniel McRobb 2025 mcplex.net
```

3.2.6 SegmentedLiteral and Info equivalents

All of the following will produce the same string, as show in [Figure 3.4](#). The first two are using macros from `DwmWhatMacros.hh` ([Table 6.1](#)) in constructor arguments. The next two are repeats of the first two, with macro substitution.

```
Dwm::What::SegmentedLiteral(DWM_WHAT_SYM_NB_SPACE, "@(#)", DWM_WHAT_TYPE_LIB,
                           DWM_WHAT_STATUS_REL, "dwmwhat", "1.0.0",
                           "Daniel McRobb", "mcplex.net");
```

```
Dwm::What::Info(DWM_WHAT_TYPE_LIB, DWM_WHAT_STATUS_REL, "dwmwhat",
               "1.0.0", "Daniel McRobb", "mcplex.net");
```

```
Dwm::What::SegmentedLiteral("\xC2\xA0", "@(#)", "\xF0\x9F\x93\x9A", "\xE2\x9C\x85",
                           "dwmwhat", "1.0.0", "Daniel McRobb", "mcplex.net");
```

```
Dwm::What::Info("\xF0\x9F\x93\x9A", "\xE2\x9C\x85", "dwmwhat",
               "1.0.0", "Daniel McRobb", "mcplex.net");
```

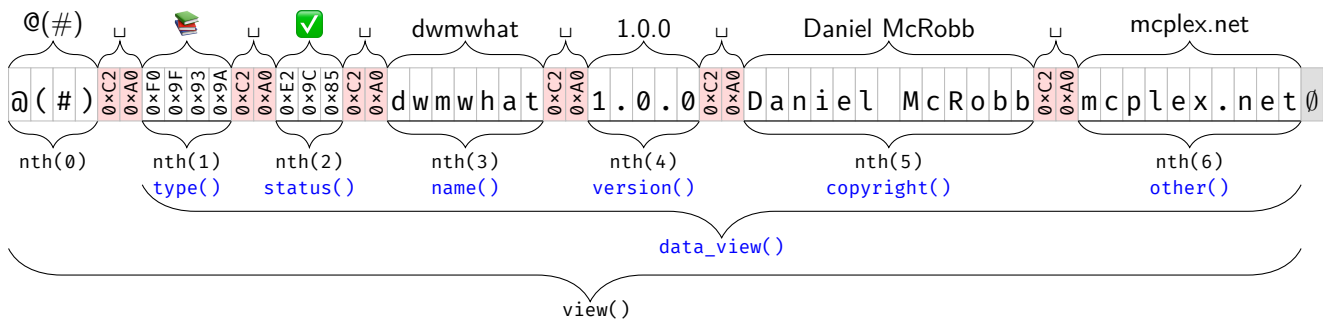


Figure 3.4: SegmentedLiteral and Info equivalents

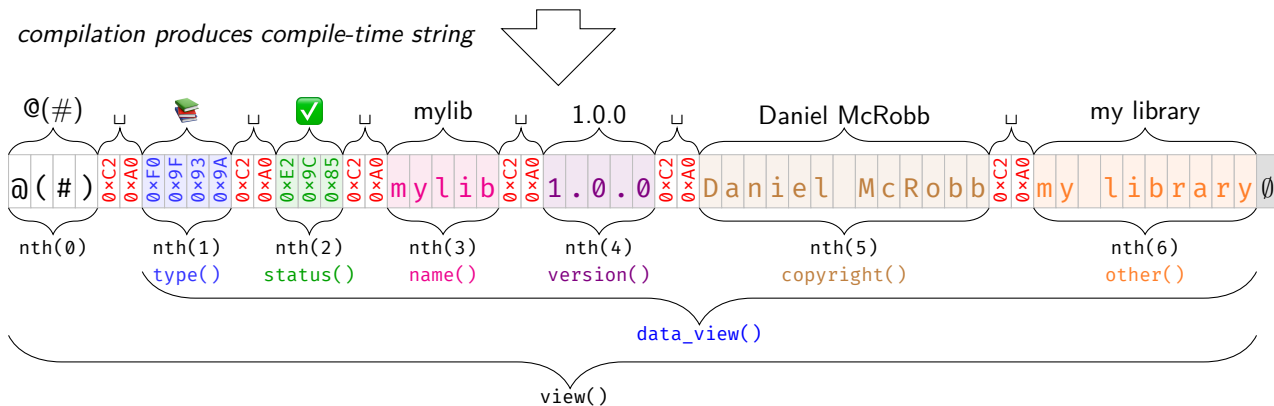
Chapter 4

More Info examples

4.1 Info Example 1

```
Dwm::What::Info(DWM_WHAT_TYPE_LIB, DWM_WHAT_STATUS_REL, "mylib", "1.0.0",  
               "Daniel McRobb", "my library")
```

compilation produces compile-time string



and an array of 7 bytes for segment lengths

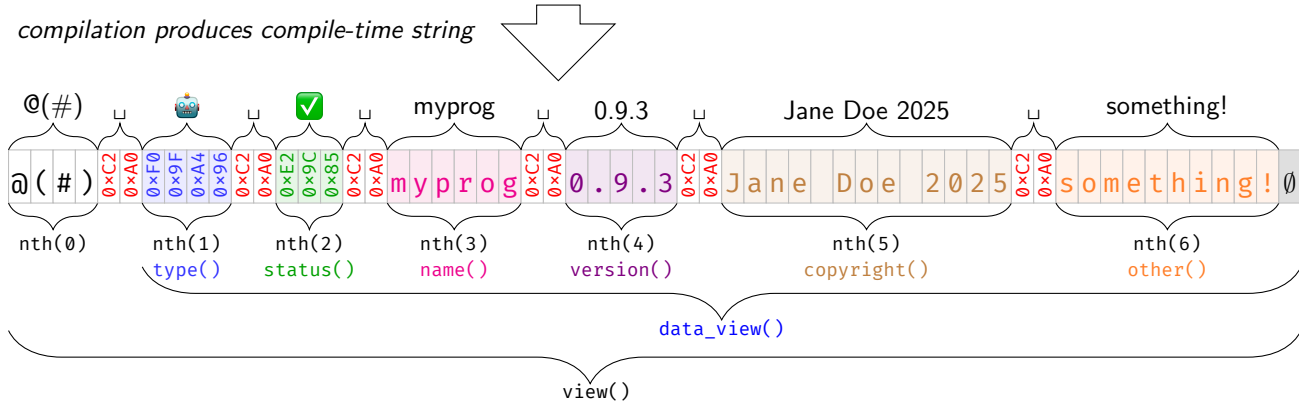
	nth(0).size()	0x04	= 4
type().size(),	nth(1).size()	0x04	= 4
status().size(),	nth(2).size()	0x03	= 3
name().size(),	nth(3).size()	0x05	= 5
version().size(),	nth(4).size()	0x05	= 5
copyright().size(),	nth(5).size()	0x0D	= 13
other().size(),	nth(6).size()	0x0A	= 10

For a total of 65 bytes.

4.2 Info Example 2

```
Dwm::What::Info(DWM_WHAT_TYPE_EXE, DWM_WHAT_STATUS_REL, "myprog", "0.9.3",
               "Jane Doe 2025", "something!")
```

compilation produces compile-time string



and an array of 7 bytes for segment lengths

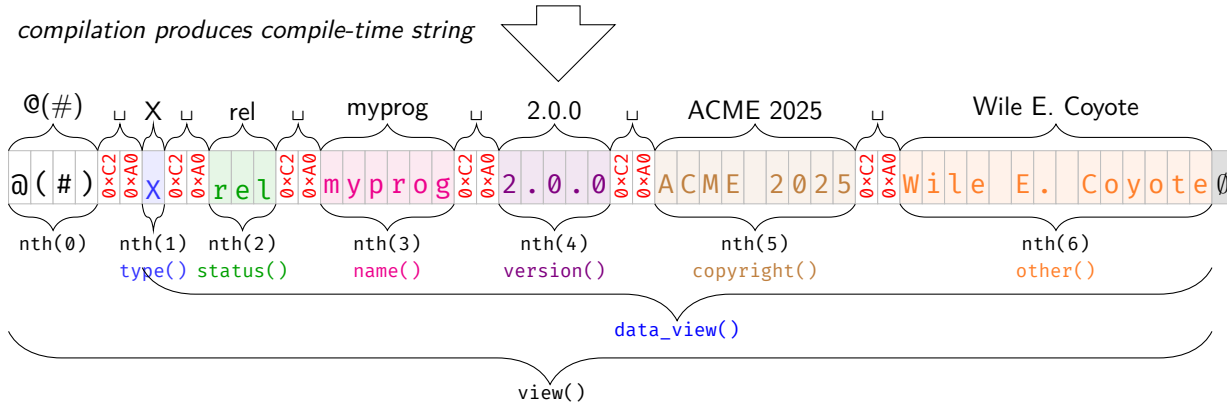
	<code>nth(0).size()</code>	<code>0x04</code>	<code>= 4</code>
<code>type().size(),</code>	<code>nth(1).size()</code>	<code>0x04</code>	<code>= 4</code>
<code>status().size(),</code>	<code>nth(2).size()</code>	<code>0x03</code>	<code>= 3</code>
<code>name().size(),</code>	<code>nth(3).size()</code>	<code>0x06</code>	<code>= 6</code>
<code>version().size(),</code>	<code>nth(4).size()</code>	<code>0x05</code>	<code>= 5</code>
<code>copyright().size(),</code>	<code>nth(5).size()</code>	<code>0x0D</code>	<code>= 13</code>
<code>other().size(),</code>	<code>nth(6).size()</code>	<code>0x0A</code>	<code>= 10</code>

For a total of 65 bytes.

4.3 Info Example 3

`Dwm::What::Info("X", "rel", "myprog", "2.0.0", "ACME 2025", "Wile E. Coyote")`

compilation produces compile-time string



and an array of 7 bytes for segment lengths

	<code>nth(0).size()</code>	<code>0x04</code>	<code>= 4</code>
<code>type().size()</code> ,	<code>nth(1).size()</code>	<code>0x01</code>	<code>= 1</code>
<code>status().size()</code> ,	<code>nth(2).size()</code>	<code>0x03</code>	<code>= 3</code>
<code>name().size()</code> ,	<code>nth(3).size()</code>	<code>0x06</code>	<code>= 6</code>
<code>version().size()</code> ,	<code>nth(4).size()</code>	<code>0x05</code>	<code>= 5</code>
<code>copyright().size()</code> ,	<code>nth(5).size()</code>	<code>0x09</code>	<code>= 9</code>
<code>other().size()</code> ,	<code>nth(6).size()</code>	<code>0x0E</code>	<code>= 14</code>

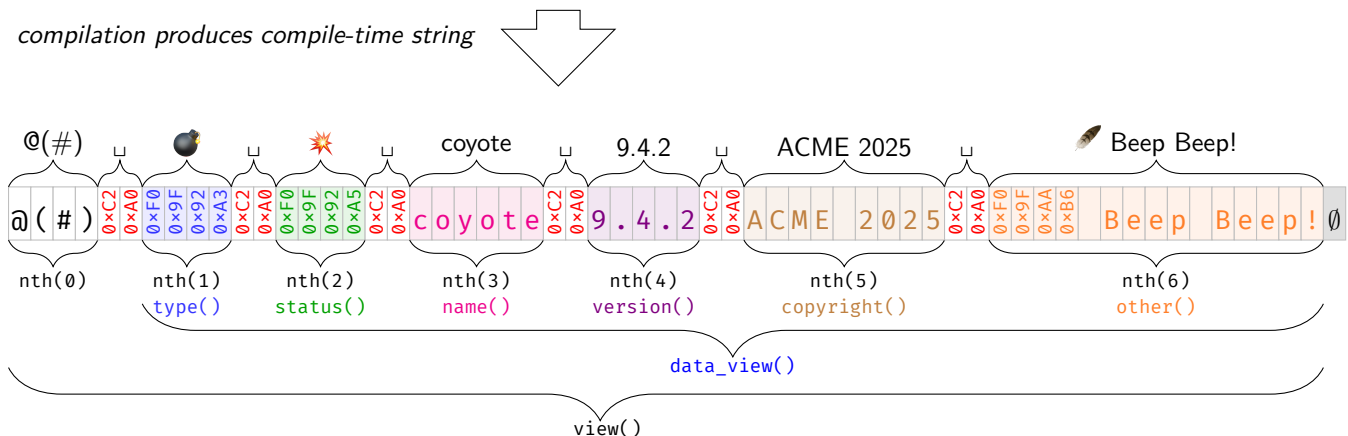
For a total of 62 bytes.

4.4 Info Example 4

If your toolchain and editor are UTF-8 aware, you can just paste emoji directly into the string literal arguments for the `Dwm::What::Info` constructor.

`Dwm::What::Info("💣", "💣", "coyote", "9.4.2", "ACME 2025", "🦉 Beep Beep!")`

compilation produces compile-time string



and an array of 7 bytes for segment lengths

	nth(0).size()	0x04	= 4
type().size(),	nth(1).size()	0x04	= 4
status().size(),	nth(2).size()	0x04	= 4
name().size(),	nth(3).size()	0x06	= 6
version().size(),	nth(4).size()	0x05	= 5
copyright().size(),	nth(5).size()	0x09	= 9
other().size(),	nth(6).size()	0x0F	= 15

For a total of 67 bytes.

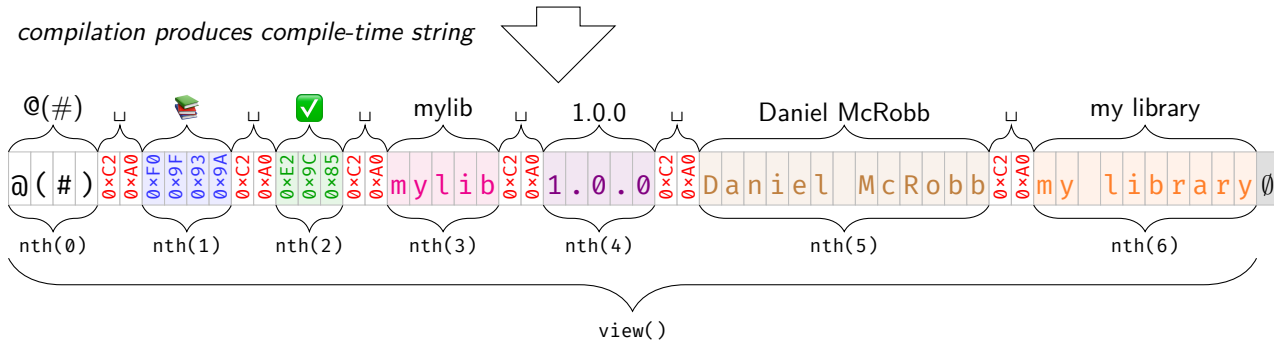
Chapter 5

More SegmentedLiteral examples

5.1 SegmentedLiteral Example 1

```
Dwm::What::SegmentedLiteral(DWM_WHAT_NB_SPACE, "@(#)", DWM_WHAT_TYPE_LIB,  
    DWM_WHAT_STATUS_REL, "mylib", "1.0.0",  
    "Daniel McRobb", "my library")
```

compilation produces compile-time string



and an array of 7 bytes for segment lengths

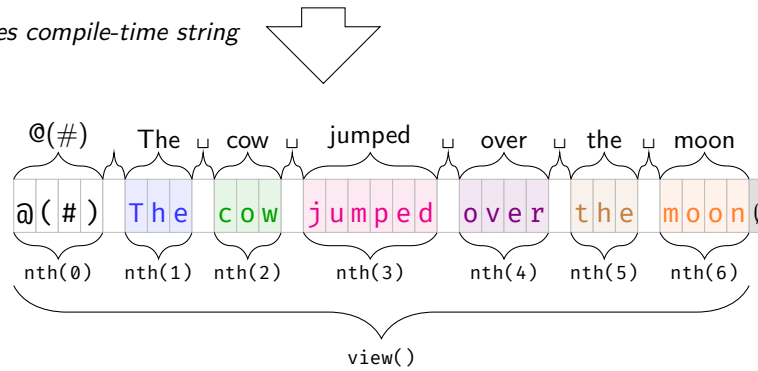
```
nth(0).size() = 0x04 = 4  
type().size(), nth(1).size() = 0x04 = 4  
status().size(), nth(2).size() = 0x03 = 3  
name().size(), nth(3).size() = 0x05 = 5  
version().size(), nth(4).size() = 0x05 = 5  
copyright().size(), nth(5).size() = 0x0D = 13  
other().size(), nth(6).size() = 0x0A = 10
```

For a total of 65 bytes.

5.2 SegmentedLiteral Example 2

`Dwm::What::SegmentedLiteral(" ", "@(#)", "The", "cow", "jumped", "over", "the", "moon")`

compilation produces compile-time string



and an array of 7 bytes for segment lengths

<code>nth(0).size()</code>	<code>0x04</code>	<code>= 4</code>
<code>nth(1).size()</code>	<code>0x03</code>	<code>= 3</code>
<code>nth(2).size()</code>	<code>0x03</code>	<code>= 3</code>
<code>nth(3).size()</code>	<code>0x06</code>	<code>= 6</code>
<code>nth(4).size()</code>	<code>0x04</code>	<code>= 4</code>
<code>nth(5).size()</code>	<code>0x03</code>	<code>= 3</code>
<code>nth(6).size()</code>	<code>0x04</code>	<code>= 4</code>

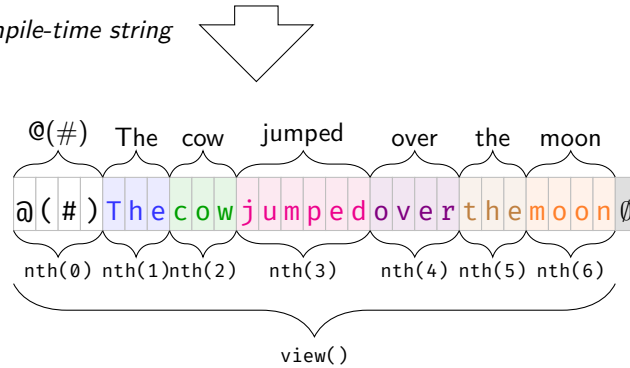
For a total of 41 bytes.

5.3 SegmentedLiteral Example 3

An empty delimiter is permitted.

Dwm::What::SegmentedLiteral("", "@(#)", "The", "cow", "jumped", "over", "the", "moon")

compilation produces compile-time string



and an array of 7 bytes for segment lengths

```

nth(0).size() { 0x04 = 4
nth(1).size() { 0x03 = 3
nth(2).size() { 0x03 = 3
nth(3).size() { 0x06 = 6
nth(4).size() { 0x04 = 4
nth(5).size() { 0x03 = 3
nth(6).size() { 0x04 = 4

```

For a total of 35 bytes: 28 for the compile-time string and 7 for the array of segment lengths.

Chapter 6

Macros in DwmWhatMacros.hh

There are many macros in `DwmWhatMacros.hh` that expand to UTF-8 string literals, just so I can get emoji output. For my own use, the macros in [Table 6.1](#) are commonly used.

category	macro	expands to (UTF-8)	Unicode Code Point	display
type	<code>DWM_WHAT_TYPE_EXE</code>	<code>"\xF0\x9F\xA4\x96"</code>	U+1F916	
	<code>DWM_WHAT_TYPE_LIB</code>	<code>"\xF0\x9F\x93\x9A"</code>	U+1F4DA	
	<code>DWM_WHAT_TYPE_HDR</code>	<code>"\xEF\xBC\x83"</code>	U+FF03	#
	<code>DWM_WHAT_TYPE_DOC</code>	<code>"\xF0\x9F\x93\x84"</code>	U+1F4C4	
status	<code>DWM_WHAT_STATUS_REL</code>	<code>"\xE2\x9C\x85"</code>	U+2705	
	<code>DWM_WHAT_STATUS_RC</code>	<code>"\xF0\x9F\x91\xB7"</code>	U+1F477	
	<code>DWM_WHAT_STATUS_DEV</code>	<code>"\xE2\x9D\x97"</code>	U+2757	!
miscellany	<code>DWM_WHAT_SYM_ALIEN</code>	<code>"\xF0\x9F\x91\xBD"</code>	U+1F47D	
	<code>DWM_WHAT_SYM_COPYRIGHT</code>	<code>"\xC2\xA9"</code>	U+00A9	©
	<code>DWM_WHAT_SYM_CYCLONE</code>	<code>"\xF0\x9F\x8C\x80"</code>	U+1F300	
	<code>DWM_WHAT_SYM_GHOST</code>	<code>"\xF0\x9F\x91\xBB"</code>	U+1F47B	
	<code>DWM_WHAT_SYM_JACKOLANTERN</code>	<code>"\xF0\x9F\x8E\x83"</code>	U+1F383	
	<code>DWM_WHAT_SYM_ROCKET</code>	<code>"\xF0\x9F\x9A\x80"</code>	U+1F680	

Table 6.1: Macros in `DwmWhatMacros.hh`